Milestone 5 Notes

Milestone 5: Building Interactive UIs with React

🔹 Web Design, Intro to Tailwind & Responsiveness

Getting Started with Tailwind, Explore Color

Tailwind CSS is a utility-first CSS framework: instead of writing custom CSS classes and rules in a separate file, you style elements by combining small, single-purpose utility classes directly in your HTML/JSX. Color utilities follow the pattern {property}-{color}-{shade}, e.g. bg-blue-500 or text-red-600, giving you a full design-system palette out of the box.

💡 **Analogy**: Think of regular CSS as cooking from scratch — you measure and mix every ingredient yourself. Tailwind is like a spice rack of pre-made utilities — you just reach for bg-blue-500, text-white, p-4 and combine them, instead of writing .my-button { background: blue; color: white; padding: 1rem; } by hand.

```
<div class="bg-blue-500 text-white p-4">
  Hello Tailwind!
</div>

<!-- bg-blue-500 = background color, shade 500 (medium) -->
<!-- text-white  = text color white -->
<!-- p-4         = padding on all sides -->
```

Tailwind Typography, Background, Text, Border

Typography utilities control font size (text-lg, text-2xl), weight (font-bold, font-medium), and alignment (text-center). Background utilities (bg-*) set background colors or images. Border utilities (border, border-2, rounded-lg) control thickness, color, and corner rounding.

```
<h1 class="text-3xl font-bold text-center text-gray-800">
 Welcome
</h1>

<div class="bg-gray-100 border-2 border-gray-300 rounded-lg p-6">
 Card content here
</div>
```

Explore Tailwind Width, Height, Padding, Margin

w-* and h-* set width/height (w-full, h-screen, w-64). Padding (p-*) adds space INSIDE an element's border; margin (m-*) adds space OUTSIDE it, between elements. Directional variants exist for both: pt- (top), pb- (bottom), px- (left+right), py- (top+bottom), and the same pattern for margin.

💡 **Analogy:** Think of a framed photo on a wall. Padding is the mat board between the photo and the frame (space inside the frame). Margin is the empty wall space between this frame and the next one (space outside the frame).

```
<div class="w-64 h-32 p-4 m-6 bg-white border rounded">
 <!-- w-64: fixed width, h-32: fixed height -->
 <!-- p-4: padding inside, m-6: margin outside -->
</div>

<div class="px-6 py-2">   <!-- padding-x and padding-y only -->
 Button text
</div>
```

Introduction to Media Query and Responsiveness

A media query applies different CSS rules depending on screen size, letting one layout adapt from mobile to desktop. Tailwind builds this in with responsive prefixes (sm:, md:, lg:, xl:) applied before any utility — the utility only activates at that breakpoint and up (a mobile-first approach).

💡 **Analogy:** Think of clothing sizes — the same 'outfit' (layout) is tailored differently for Small, Medium, and Large bodies (screens). Tailwind's sm:/md:/lg: prefixes are exactly that: different fits of the same design for different screen sizes.

```
<div class="text-sm md:text-lg lg:text-2xl">
 <!-- small text on mobile, larger on tablet (md), largest on desktop (lg) -->
 Responsive Heading
```

```
</div>

<div class="w-full md:w-1/2 lg:w-1/3">
  <!-- full width on mobile, half on tablet, a third on desktop -->
</div>
```

Tailwind Flex Layout and Responsive Flex

flex turns an element into a flex container, arranging its children along a single axis (row by default). flex-col switches to a vertical axis. justify-* controls alignment along the main axis (justify-between, justify-center), and items-* controls alignment along the cross axis (items-center). Combine with responsive prefixes to stack on mobile and go horizontal on desktop.

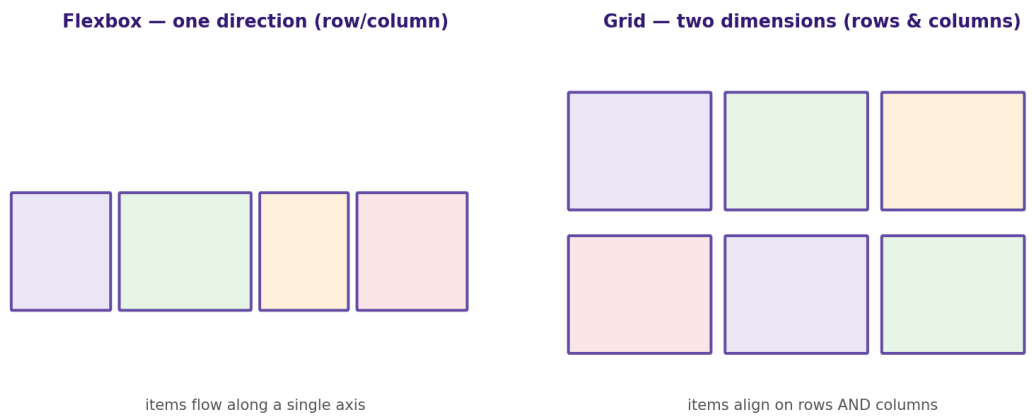
💡 **Analogy:** Flexbox is like arranging books on a single shelf — you can push them to one end, spread them evenly, or center them, but they're all still in one row (or one column).

```
<div class="flex flex-col md:flex-row justify-between items-center gap-4">
  <div>Logo</div>
  <div>Nav Links</div>
  <div>Button</div>
</div>
<!-- stacked vertically on mobile, side-by-side on tablet+ -->
```

Explore Grid Layout and Responsive Grid

grid turns an element into a grid container; grid-cols-* defines how many equal-width columns it has. Unlike flex (one axis), grid naturally arranges items in both rows and columns, making it ideal for card layouts and galleries. gap-* adds space between grid cells, and responsive prefixes change the column count per breakpoint.

💡 **Analogy:** If Flexbox is a single shelf, Grid is a full bookcase — items are organized into both rows AND columns at once, like a spreadsheet or graph paper.




```
<div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
  <div class="bg-white p-4 rounded shadow">Card 1</div>
  <div class="bg-white p-4 rounded shadow">Card 2</div>
  <div class="bg-white p-4 rounded shadow">Card 3</div>
</div>
<!-- 1 column on mobile, 2 on tablet, 3 on desktop -->
```

🖋️ Tailwind Typography, Background, Text & Border

📌 Typography

Class	Purpose
text-xs → text-9xl	Font sizes
font-light → font-black	Font weights
text-center, text-left	Alignment

tracking-wider, leading-loose	Letter/line spacing
uppercase, capitalize	Text case

```
<h1 class="text-3xl font-bold text-center uppercase">Hello</h1>
```

🎨 Backgrounds

Class	Description
bg-red-100 to bg-red-900	Background colors
bg-gradient-to-r from-indigo-500 to-pink-500	Gradients

✍️ Borders

Class	Description
border, border-2, border-t	Sides and thickness

border-gray-300	Color
rounded, rounded-lg, rounded-full	Radius styles

📦 Tailwind Spacing & Box Model

Margin & Padding

- m, mx, my, mt, mb, ml, mr
- p, px, py, etc.
- Numeric scale: 0 → 96 (spacing unit = 0.25rem)

Content

Width/Height

Class	Example
w-full, w-1/2, w-96	Width

h-64, min-h-screen	Height

👓Tailwind Flex Layout + Responsive Flex

  Item 1

  Item 2

🧱 Flex Utilities:

Class	Description
flex, inline-flex	Display
flex-row, flex-col	Direction
justify-*	Main axis
items-*	Cross axis

gap-*	Item spacing
flex-wrap, basis-1/2, grow, shrink	Advanced control

🔲 Tailwind Grid Layout + Responsive Grid

```
<div class="grid grid-cols-1 md:grid-cols-3 gap-6">
  <div class="col-span-2">Wide</div>
  <div>Box</div>
</div>
```

🌐 Grid Utilities:

Class	Description
grid-cols-*	Columns
grid-rows-*	Rows
col-span-*, row-span-*	Item span

gap, place-items, justify-items	Spacing/alignment

🔁 Responsive Grid Layout:

grid-cols-1 sm:grid-cols-2 md:grid-cols-3

🎯State-Based Classes (hover, focus, active, peer)

Prefix	Description
hover:	On mouse hover
focus:	On input focus
active:	On button click
disabled:	When disabled
group-hover:	Parent element hover
peer-focus:	Related sibling focus

<input class="peer ...">

Focus me!

🔁 **Dynamic classes:**

<button class="bg-blue-500 hover:bg-blue-700 focus:ring-2 focus:ring-blue-400">Click</button>

🔹 React Core: Components, JSX, Props & Rendering

What Is React? React vs Next.js (in TypeScript)

React is a JavaScript library for building user interfaces out of reusable, self-contained components — it focuses purely on the 'view' layer and lets you choose your own tools for routing, data fetching, etc. Next.js is a framework built ON TOP of React that adds routing, server-side rendering, and other production features out of the box.

💡 **Analogy**: React is like a box of high-quality LEGO bricks — powerful, flexible, but you decide how to assemble the whole house yourself. Next.js is like a LEGO starter kit that already includes a pre-built frame, plumbing, and wiring (routing, SSR, build config) around those same bricks — less assembly required, more opinions baked in.

What Is a Component? Component Types, Build Your First Component

A component is a reusable, self-contained piece of UI — a function that returns JSX describing what should appear on screen. Modern React uses function components almost exclusively (older code sometimes used class components). Each component can be reused anywhere, like a custom HTML tag you invented yourself.

💡 **Analogy**: A component is like a cookie cutter — define the shape once, then stamp out as many cookies (UI instances) as you need, each one independent.

```
// A simple function component in TypeScript
function Welcome(): JSX.Element {
 return <h1>Hello, welcome to React!</h1>;
```

```
}

export default Welcome;

// Used elsewhere:
// <Welcome />
```

JSX with TypeScript — Its Rules and Adding Dynamic Content

JSX lets you write HTML-like syntax directly inside JavaScript/TypeScript, which gets compiled into regular function calls under the hood. Key rules: every JSX block must return a single root element (wrap multiple elements in a or a <> fragment), use className instead of class, and close every tag (including self-closing tags like <img />). Wrap any JavaScript expression in curly braces {} to inject dynamic content.

💡 Analogy: JSX is like a mail-merge template — the HTML-looking structure is the fixed template, and the {curly braces} are the blanks that get filled in with real data at render time.

```
function Greeting(): JSX.Element {
 const name: string = "Alice";
 const hour: number = 14;
 return (
  <div className="greeting">
   <h2>Hello, {name}!</h2>
   <p>{hour < 12 ? "Good morning" : "Good afternoon"}</p>
  </div>
 );
}
```

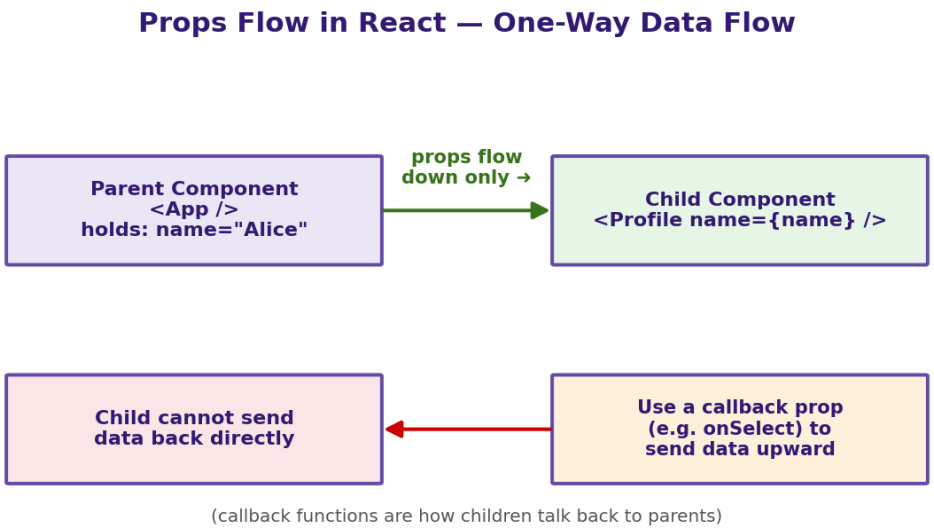
Explore JSX and Props Basics

Props (short for 'properties') are how data is passed INTO a component from its parent, similar to how arguments are passed into a function. Props make a component configurable and reusable, since the same component can render differently depending on what props it receives.

How to Pass and Read Props in React

Props are passed like HTML attributes when a component is used, and read inside the component via its parameter (typically destructured). In TypeScript, you define an interface or type describing exactly which props a component accepts and their types.

💡 Analogy: Props are like a sealed, addressed envelope handed from parent to child — the parent decides exactly what goes inside, and the child can only read the contents, never reseal or redirect the envelope back.




```
interface GreetingProps {
  name: string;
  age: number;
}

function Greeting({ name, age }: GreetingProps): JSX.Element {
  return <p>{name} is {age} years old.</p>;
}

// Parent passes props like HTML attributes:
// <Greeting name="Alice" age={25} />
```

Read-Only Props & Introduction to Conditional Rendering

Props are read-only — a component must NEVER modify its own props directly; doing so breaks React's data flow and causes unpredictable bugs. Conditional rendering means showing different JSX depending on a condition (like a prop's value or component state), similar to an if statement but for UI output.

```
interface StatusProps { isOnline: boolean; }
```

```
function StatusBadge({ isOnline }: StatusProps): JSX.Element {
 // isOnline is read ONLY, never reassigned here
 if (isOnline) {
  return <span className="text-green-600">🟢 Online</span>;
 }
 return <span className="text-gray-400">⚪ Offline</span>;
}
```

Different Ways to Do Conditional Rendering: If, Ternary, AND/OR

There are three common patterns: a full if/else (usually before the return, or inside a helper function) for more complex branching; the ternary operator (condition ? A : B) for a compact inline either/or; and the && operator to render something only when a condition is true (renders nothing — not even an empty tag — when false).

```
// 1. if/else (before return)
function Badge({ isOnline }: { isOnline: boolean }) {
 if (isOnline) return <span>🟢 Online</span>;
 return <span>⚪ Offline</span>;
}

// 2. Ternary — inline either/or
function Badge2({ isOnline }: { isOnline: boolean }) {
 return <span>{isOnline ? "🟢 Online" : "⚪ Offline"}</span>;
}

// 3. && — render only if true
function Notification({ hasAlert }: { hasAlert: boolean }) {
 return <div>{hasAlert && <span className="text-red-500">⚠ New Alert</span>}</div>;
}
```

(Advanced) Rendering Lists of Users Using map

To render a list of items, use the array's .map() method to transform each data item into a JSX element, returning an array of elements that React renders in order. Every item in the list needs a unique key prop so React can efficiently track which items changed, were added, or were removed between renders.

💡 **Analogy**: .map() here works like a factory assembly line — raw data goes in one end (the array), and a finished, labeled product (a JSX element with a unique key tag) comes out the other end for every single item.

```
interface User { id: number; name: string; }
const users: User[] = [
 { id: 1, name: "Alice" },
 { id: 2, name: "Bob" }
];

function UserList(): JSX.Element {
 return (
  <ul>
   {users.map(user => (
    <li key={user.id}>{user.name}</li>
   ))}
  </ul>
 );
}
```

🔹 State Management in React, Events & Data Fetching

Vite Project Setup: Folder Structure, Files, Environment

Vite is a fast modern build tool for starting React (and other) projects, replacing the slower Create React App. A typical Vite + React project has: src/ (your components and code), public/ (static assets), index.html (the single page the app mounts into), main.tsx (the entry point that renders your root App component), and vite.config.ts (build configuration).

```
// Create a new Vite + React + TypeScript project
npm create vite@latest my-app -- --template react-ts
cd my-app
npm install
npm run dev

// Typical structure:
// src/
//   main.tsx     ← entry point
//   App.tsx      ← root component
//   components/  ← your reusable components
```

Event Handling: onClick, onChange, onSubmit and Patterns

React attaches event handlers directly in JSX using camelCase prop names (onClick, onChange, onSubmit) set to a function — not a string like in plain HTML. The handler function automatically receives an event object with details about what happened.

```
function Form(): JSX.Element {
 function handleClick(): void {
  console.log("Button clicked!");
 }
 function handleChange(e: React.ChangeEvent<HTMLInputElement>): void {
  console.log("Input value:", e.target.value);
 }
 function handleSubmit(e: React.FormEvent): void {
  e.preventDefault(); // stop the page from reloading
  console.log("Form submitted");
 }
 return (
  <form onSubmit={handleSubmit}>
   <input onChange={handleChange} />
   <button onClick={handleClick} type="button">Click Me</button>
  </form>
 );
}
```

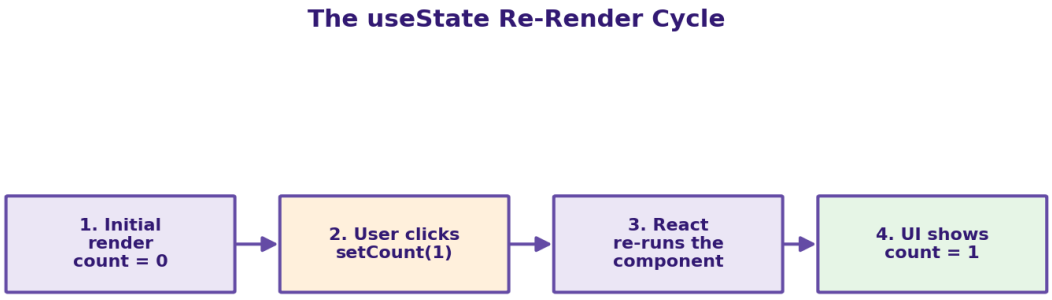
State in React: Why State Matters, How Re-Rendering Works

State is data that a component owns and manages internally, which can change over time (unlike props, which come from outside and are read-only). Whenever state changes, React automatically re-runs that component's function to recompute its JSX, then efficiently updates only the parts of the real DOM that actually changed.

💡 **Analogy:** Think of state like the number shown on a car's speedometer — the dashboard (UI) automatically redraws the needle position the instant the underlying speed (state) changes, without you manually repainting the whole dashboard.

useState Hook: Declaring, Updating, and Reading State

useState is the hook that gives a function component its own piece of state. It returns a pair: the current value, and a setter function to update it. Calling the setter schedules a re-render with the new value — you never modify the state variable directly.




```
import { useState } from "react";

function Counter(): JSX.Element {
 const [count, setCount] = useState<number>(0); // [currentValue, setterFunction]

 return (
  <div>
   <p>Count: {count}</p>
   <button onClick={() => setCount(count + 1)}>+1</button>
   <button onClick={() => setCount(count - 1)}>-1</button>
  </div>
 );
}
```

What Is an API? How to Use It?

An API (Application Programming Interface) is a defined way for one piece of software to request data or services from another — in web development, this usually means your frontend sending an HTTP request to a server and getting data back (often as JSON). The browser's built-in fetch() function is the standard way to make these requests.

💡 **Analogy:** An API is like a restaurant waiter — you (the frontend) don't walk into the kitchen (the server/database) yourself. You place an order (a request) with the waiter (the API), and the waiter brings back exactly what you asked for (the response), without you needing to know how the kitchen works internally.

```
// Basic fetch example
fetch("https://api.example.com/users")
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error("Error:", error));
```

The use() Hook: Modern Data-Fetching Hook

use() is a newer React hook that lets you read the value of a Promise (or Context) directly during render, pausing the component until the data resolves — it works together with React's Suspense feature to show a fallback UI while waiting. It simplifies some data-fetching patterns that previously required useEffect + useState combined.

```
import { use } from "react";

function UserProfile({ userPromise }: { userPromise: Promise<{ name: string }> }) {
  const user = use(userPromise); // suspends until the promise resolves
  return <p>Welcome, {user.name}</p>;
}
// Must be wrapped in <Suspense fallback={<p>Loading...</p>}> by a parent
```

Async/Await in React: Fetching Data with Async Functions

async/await is a cleaner, more readable syntax over raw .then() chains for working with Promises (like the result of fetch()). An async function always returns a Promise, and await pauses execution inside that function until the Promise resolves.

```
async function getUsers(): Promise<void> {
  try {
    const response = await fetch("https://api.example.com/users");
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error("Failed to fetch:", error);
  }
}
```

useEffect Hook: Side Effects, Dependencies

useEffect runs code in response to a component rendering or specific values changing — typically used for things outside React's normal rendering, like fetching data, setting up subscriptions, or manually interacting with the DOM ('side effects'). Its second argument, the dependency array, controls when it re-runs: empty [] means 'run once on mount,' a

list of values means 're-run whenever any of these change,' and omitting it entirely means 'run after every render.'

💡 Analogy: useEffect is like a security guard watching a specific set of doors (the dependency array). The guard only takes action when one of those exact doors opens (a dependency changes) — with an empty list of doors to watch, the guard only acts once, right when they start their shift.

```
import { useState, useEffect } from "react";

function UserList(): JSX.Element {
 const [users, setUsers] = useState<string[]>([]);

 useEffect(() => {
  fetch("https://api.example.com/users")
   .then(res => res.json())
   .then(data => setUsers(data));
 }, []); // empty array = run once, when the component first mounts

 return <ul>{users.map((u, i) => <li key={i}>{u}</li>)}</ul>;
}
```

Thinking in React: Component Mindset

'Thinking in React' means breaking a UI mockup into a tree of components before writing any code: identify what should be its own component (anything reused, or any distinct piece of UI/data), decide which component should own which piece of state (state should live in the closest common parent of everything that needs it), and build static versions first before wiring up interactivity. This planning step saves far more time than it costs.

🔹 React Project: Countries Explorer

Project Setup: Project Structure and Data Fetching

The project starts by setting up a Vite + React + TypeScript app, then fetching country data from a public REST API (like the REST Countries API) inside a useEffect, storing the result in state so the UI can render once the data arrives.

```
interface Country { name: { common: string }; flags: { png: string }; population: number; }

function App(): JSX.Element {
 const [countries, setCountries] = useState<Country[]>([]);
```

```
 useEffect(() => {
  fetch("https://restcountries.com/v3.1/all")
    .then(res => res.json())
    .then(data => setCountries(data));
 }, []);

 return <CountryList countries={countries} />;
}
```

UI Rendering: Display Country List

Once the country data is in state, render it by mapping over the array and passing each country's data down as props to a reusable CountryCard component — the list component's job is only to loop and delegate; the card component's job is to display one country.

Country Card: Show Flags, Conditional Text

Each CountryCard displays the flag image and name, and uses conditional rendering for details that might be missing (like showing 'Population data unavailable' if the field is undefined) — reinforcing conditional rendering with real, messy API data.

```
function CountryCard({ country }: { country: Country }): JSX.Element {
 return (
  <div className="border rounded p-4">
   <img src={country.flags.png} alt={country.name.common} className="w-full h-32
object-cover" />
   <h3 className="font-bold mt-2">{country.name.common}</h3>
   <p>{country.population ? country.population.toLocaleString() : "Population unknown"}</p>
  </div>
 );
}
```

React Styling: Styling with CSS

Alongside (or instead of) Tailwind, components can be styled with plain CSS files imported directly into a component, or with CSS Modules (styles scoped to just that component) to avoid class name collisions across a growing app.

Responsive Grid: Build a 3-Column Layout

The country list uses a responsive CSS grid so it shows 1 column on mobile, 2 on tablet, and 3 on desktop, keeping each card a comfortable, readable size at every screen width.

Interactive UI: Conditional CSS Classes and Toggle State

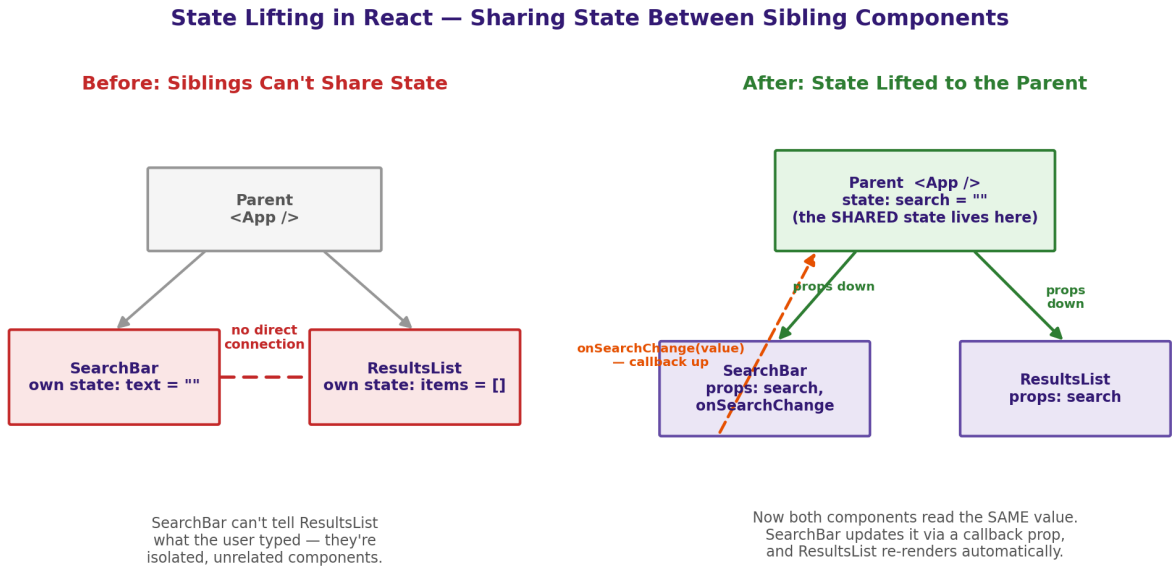
A common pattern is toggling a piece of boolean state (like isFavorite) with useState, then conditionally applying a different CSS class based on that state — giving instant visual feedback (like a highlighted border) without touching the DOM manually.

```
function CountryCard({ country }: { country: Country }): JSX.Element {
 const [isFavorite, setIsFavorite] = useState(false);
 return (
  <div className={isFavorite ? "border-4 border-yellow-400" : "border"}>
   <button onClick={() => setIsFavorite(!isFavorite)}>
    {isFavorite ? "★ Favorited" : "☆ Favorite"}
   </button>
  </div>
 );
}
```

State Lifting: Pass State and Events Between Components

State lifting means moving a piece of state UP to the nearest shared parent when two or more sibling components need to read or affect the same value — the parent then passes the state down as a prop, and passes a callback function down so children can request changes to it.

💡 **Analogy:** If two children share one toy, you don't give each child their own separate copy — you give the toy to the parent (the shared state), and both kids ask the parent whenever they want a turn (via a callback prop).




```
function App(): JSX.Element {
 const [search, setSearch] = useState("");
 return (
  <>
   <SearchBar search={search} onSearchChange={setSearch} />
   <CountryList search={search} />
  </>
 );
}
// Both SearchBar and CountryList share the same 'search' state via App
```

Immutable State: Update Arrays Without Mutation

React needs a NEW array/object reference to detect that state changed and trigger a re-render — directly mutating the existing array (like .push() on state) won't reliably update the UI. Instead, always create a new array using spread, map, or filter when updating array state.

```
// ❌ Wrong — mutates the existing array in place
function addCountry(newCountry: Country) {
 countries.push(newCountry);   // React may not detect this change
 setCountries(countries);
}
```

```
// ✅ Correct — creates a brand-new array
function addCountry(newCountry: Country) {
  setCountries([...countries, newCountry]);
}

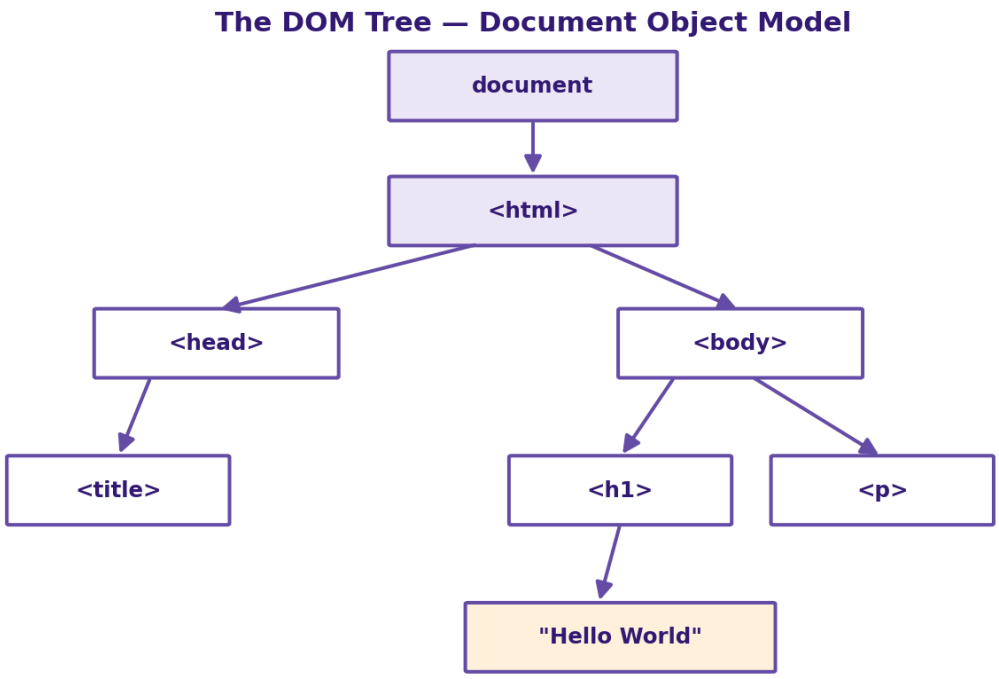
// ✅ Removing an item immutably
function removeCountry(id: string) {
  setCountries(countries.filter(c => c.id !== id));
}
```

🔹 JavaScript DOM & BOM Deep Dive (React Bridge)

DOM Basics: Document Object Model and Its Tree Structure

The DOM is the browser's in-memory, tree-shaped representation of an HTML page — every tag becomes a 'node' with parent/child relationships, which JavaScript can read and modify. Understanding the DOM directly makes it much easier to understand what React is actually doing underneath its abstractions.

💡 **Analogy:** The DOM is like a family tree, but upside down — document is the ancestor at the top, and every HTML tag is a descendant node branching downward, each with its own children.



DOM Selection: getElement Methods vs querySelector

The older getElementById / getElementsByClassName / getElementsByTagName methods select elements by one specific criterion and (for the plural ones) return a 'live' HTMLCollection. The modern querySelector / querySelectorAll accept any standard CSS selector string and are far more flexible, returning the first match or a static NodeList.

```
// Older methods
const title = document.getElementById("title");
const items = document.getElementsByClassName("item");

// Modern, flexible — accepts any CSS selector
const firstButton = document.querySelector(".btn-primary");
const allButtons = document.querySelectorAll("button.active");
```

DOM Manipulation: Changing Text, Attributes, and Styling

Once an element is selected, you can change its visible text (textContent or innerText), its HTML content (innerHTML — use carefully, it can introduce security risks with untrusted input), its attributes (setAttribute(), or direct properties like .src), and its inline styles (.style.property).

```
const heading = document.querySelector("h1")!;
heading.textContent = "Updated Title";
heading.style.color = "blue";

const img = document.querySelector("img")!;
img.setAttribute("src", "new-image.jpg");
```

Event Handling: addEventListener and Common Event Types

addEventListener attaches a function to run when a specific event occurs on an element, without overwriting any other handlers already attached (unlike setting element.onclick = ... directly). Common event types include click, input, change, submit, keydown, and mouseover.

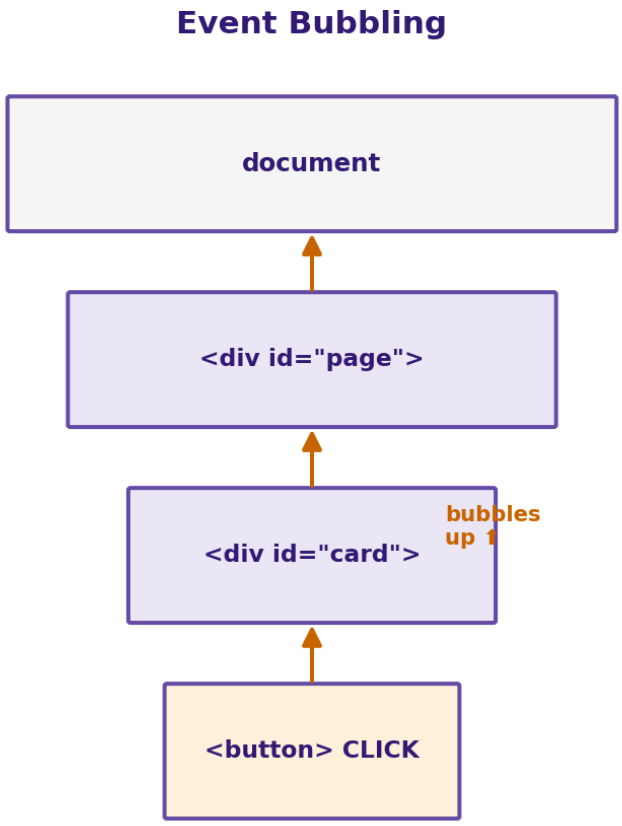
```
const button = document.querySelector("button")!;
button.addEventListener("click", () => {
  console.log("Button was clicked!");
});

document.addEventListener("keydown", (e) => {
  if (e.key === "Enter") console.log("Enter pressed");
});
```

Event Flow: Bubbling, Capturing, and Event Delegation

When an event fires on an element, it travels through three phases: capturing (top-down, from document to the target), the target phase, then bubbling (bottom-up, from the target back to document) — bubbling is the default most developers interact with. Event delegation exploits bubbling by attaching ONE listener to a parent element instead of many listeners on every child, checking event.target to determine which child was actually interacted with.

💡 **Analogy:** Event bubbling is like shouting in a crowded room — your shout (the event) doesn't stay only where you're standing; it travels outward through everyone around you (the ancestor elements) until it reaches the edges of the room (document).



Click starts at the button, then "bubbles" upward through every ancestor element, unless .stopPropagation() is called.

```
// Event delegation — one listener handles every button inside #list
document.querySelector("#list")!.addEventListener("click", (e) => {
  const target = e.target as HTMLElement;
  if (target.tagName === "BUTTON") {
    console.log("Clicked button:", target.textContent);
  }
});
```

BOM Essentials: Window Object, Location, and History API

The BOM (Browser Object Model) is everything the browser exposes BEYOND the page content itself — window is the global object representing the browser tab;

window.location holds the current URL and lets you redirect; window.history lets you navigate programmatically through the tab's back/forward session history.

```
console.log(window.innerWidth);     // viewport width
window.location.href = "https://example.com"; // redirect
console.log(window.location.pathname);         // current path

window.history.back();     // go to previous page
window.history.pushState({}, "", "/new-path"); // change URL without reload
```

Timing in JavaScript: setTimeout, setInterval, and clearTimeout

setTimeout runs a function once, after a specified delay (in milliseconds). setInterval runs a function repeatedly, every specified interval, until stopped. clearTimeout / clearInterval cancel a pending timeout/interval using the ID returned when it was created — essential for cleaning up timers (especially inside React's useEffect) to avoid memory leaks.

```
const timeoutId = setTimeout(() => console.log("3 seconds passed"), 3000);
clearTimeout(timeoutId); // cancels it before it fires

const intervalId = setInterval(() => console.log("tick"), 1000);
setTimeout(() => clearInterval(intervalId), 5000); // stop after 5 seconds
```

Browser Storage: localStorage, sessionStorage — Read, Write, Delete

localStorage persists key-value string data in the browser with no expiration (survives closing the tab/browser). sessionStorage works the same way but is cleared when the tab is closed. Both only store strings, so objects/arrays must be converted with JSON.stringify() before saving and JSON.parse() after reading.

```
// Write
localStorage.setItem("user", JSON.stringify({ name: "Alice" }));

// Read
const raw = localStorage.getItem("user");
const user = raw ? JSON.parse(raw) : null;

// Delete one key, or everything
localStorage.removeItem("user");
localStorage.clear();
```

Async JavaScript: Promise, fetch API, and the React Connection

A Promise represents a value that isn't available yet but will be resolved (success) or rejected (failure) at some point — fetch() returns a Promise, which is why it's used with .then()/.catch() or async/await. This is exactly the mechanism React's useEffect and the use() hook build on top of when fetching data for a component.

Vanilla JS vs React: Virtual DOM, Diffing, and the Component Mindset

In vanilla JavaScript, you manually select elements and mutate the real DOM directly — fast for small changes, but error-prone and hard to keep organized as an app grows. React instead keeps a lightweight in-memory copy of the UI (the Virtual DOM); whenever state changes, it builds a new Virtual DOM tree, compares ('diffs') it against the previous one, and applies only the minimal necessary changes to the real DOM.

💡 Analogy: Manually updating the real DOM is like repainting an entire wall every time one small spot gets dirty. React's Virtual DOM diffing is like using a tracked-changes document — it compares the new version against the old one and only touches the specific words that actually changed.

